

Representação Numérica e Erro

Sumário

1. Representação Numérica	3
1.1. Sistema Decimal (Base 10)	3
1.2. Sistema Binário (Base 2)	4
1.3. Sistema Hexadecimal (Base 16)	5
1.4. Comparação entre as Bases	6
1.5. Conversões Passo a Passo	7
1.5.1. Decimal → Binário	7
1.5.2. Binário → Decimal	7
1.5.3. Decimal → Hexadecimal	8
1.5.4. Hexadecimal → Decimal	8
1.5.5. Binário → Hexadecimal	9
1.5.6. Hexadecimal → Binário	10
1.6. Exercícios Propostos	10
2. Representação de Números em Computadores	11
2.1. Representação de Números Inteiros	11
2.1.1. Números Inteiros Sem Sinal (<i>Unsigned</i>)	11
2.1.2. Números Inteiros Com Sinal (<i>Signed</i>)	11
2.2. Tipos Inteiros no Octave	12
2.2.1. int8 - Inteiro de 8 bits com Sinal	12
2.2.2. int16 - Inteiro de 16 bits com Sinal	12
2.2.3. int32 - Inteiro de 32 bits com Sinal	13
2.2.4. int64 - Inteiro de 64 bits com Sinal	13
2.2.5. Comando whos para Verificar Tipos	13
2.3. Tabela Comparativa de Tipos Inteiros	14
2.4. Representação de Números Negativos	14
2.4.1. Complemento de 2 - O Padrão Moderno	14
2.4.2. Como Converter um Número para Seu Negativo em Complemento de 2	14
2.4.5. Verificação em Octave	15
2.5. Representação de Números em Ponto Flutuante	15
2.5.1. Padrão IEEE 754	15

2.5.2.	Float (Precisão Simples - 32 bits)	16
2.5.3.	Double (Precisão Dupla - 64 bits).....	16
2.5.4.	Comparação Float vs Double	17
2.6.	Soma Binária.....	17
2.7.	Exercícios Propostos	19
3.	Erros Numéricos em Cálculo Numérico	20
3.1.	Classificação de Erros	20
3.2.	Visualização dos Erros no Processo Computacional	21
3.3.	Erro Absoluto e Erro Relativo	21
3.3.1.	Erro Absoluto.....	21
3.3.2.	Erro Relativo	21
3.3.3.	Comparação Erro Absoluto vs Relativo	22
3.3.4.	Erro Relativo Percentual Aproximado	22
3.4.	Erro de Truncamento	23
3.5.	Erro de Arredondamento.....	24
3.5.1.	Regras de Arredondamento	25
3.6.	Erro de Cancelamento Catastrófico	25
3.6.1.	Identificando Cancelamento Catastrófico	26
3.7.	Propagação de Erros.....	26
3.8.	Acurácia vs Precisão.....	27
3.9.	Exemplos Práticos em Octave.....	29
3.9.1.	Programa: Calculando Erros Absoluto e Relativo.....	29
3.9.2.	Programa: Erro de Truncamento em Série de Taylor	30
3.9.3.	Programa: Demonstrando Cancelamento Catastrófico	32
3.10.	Exercícios Propostos	33

1. Representação Numérica

Definição:

Uma **base numérica** é um sistema de numeração que usa um conjunto de símbolos (dígitos) para representar valores. A base define **quantos símbolos diferentes** existem.

Valor Posicional:

Em qualquer base, o valor de um número depende:

- Do dígito** (símbolo usado)
- Da posição** (potência da base)

Fórmula Geral de Conversão

$$N = d_n \times b^n + d_{n-1} \times b^{n-1} + \dots + d_1 \times b^1 + d_0 \times b^0$$

Onde:

- N** = valor decimal
- d** = dígito em cada posição
- b** = base do sistema
- n** = posição (começando do 0 à direita)

1.1. Sistema Decimal (Base 10)

Aspecto	Descrição
Base	10
Dígitos	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Uso	Cotidiano, matemática, ciência
Total de Símbolos	10 dígitos

Valor Posicional no Decimal

Cada posição representa uma **potência de 10**:

Posição	Valor	Resultado
0	10^0	1
1	10^1	10
2	10^2	100
3	10^3	1000

Exemplo 1: Analisando o Número 5.347

$$\begin{aligned} 5.347 &= 5 \times 10^3 + 3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0 \\ &= 5 \times 1000 + 3 \times 100 + 4 \times 10 + 7 \times 1 \\ &= 5000 + 300 + 40 + 7 \\ &= 5.347 \end{aligned}$$

Decomposição Visual:

Milhares: $5 \times 1000 = 5000$

Centenas: $3 \times 100 = 300$

Dezenas: $4 \times 10 = 40$

Unidades: $7 \times 1 = 7$

5347

Exemplo 2: Analisando o Número 842

$$\begin{aligned} 842 &= 8 \times 10^2 + 4 \times 10^1 + 2 \times 10^0 \\ &= 8 \times 100 + 4 \times 10 + 2 \times 1 \\ &= 800 + 40 + 2 \\ &= 842 \end{aligned}$$

1.2. Sistema Binário (Base 2)

Aspecto	Descrição
Base	2
Dígitos	0, 1 (chamados de bits)
Uso	Computadores, eletrônica digital
Total de Símbolos	2 dígitos

Os computadores usam **dois estados elétricos**:

- **0** = sem corrente elétrica (desligado)
- **1** = com corrente elétrica (ligado)

Valor Posicional no Binário

Cada posição representa uma **potência de 2**:

Posição	Valor	Resultado
0	2^0	1
1	2^1	2
2	2^2	4
3	2^3	8
4	2^4	16
5	2^5	32
6	2^6	64
7	2^7	128

Exemplo 1: Convertendo $(10110)_2$ para Decimal

$$\begin{aligned} 10110_2 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ &= 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 \\ &= 16 + 0 + 4 + 2 + 0 \\ &= 22_{10} \end{aligned}$$

Representação Visual:

Posição: 4 3 2 1 0
Bit: 1 0 1 1 0
Valor: 16 0 4 2 0

Total (decimal): 22

Exemplo 2: Convertendo $(11111111)_2$ para Decimal

$$\begin{aligned} 11111111_2 &= 1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 \\ &= 255_{10} \end{aligned}$$

1.3. Sistema Hexadecimal (Base 16)

Aspecto	Descrição
Base	16
Dígitos	0-9 e A-F
Uso	Programação, cores (RGB), endereços de memória
Total de Símbolos	16 dígitos

Tabela de Dígitos Hexadecimais

Decimal	Hexadecimal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	A
11	B
12	C
13	D
14	E
15	F

Valor Posicional no Hexadecimal

Cada posição representa uma **potência de 16**:

Posição	Valor	Resultado
3	16^3	4096
2	16^2	256
1	16^1	16
0	16^0	1

Exemplo 1: Convertendo $(3AB)_{16}$ para Decimal

$$3AB_{16} = 3 \times 16^2 + A \times 16^1 + B \times 16^0$$

Substituindo as letras:

$$3AB_{16} = 3 \times 256 + 10 \times 16 + 11 \times 1$$

$$3AB_{16} = 768 + 160 + 11$$

$$3AB_{16} = 939_{10}$$

Representação Visual:

Posição	Dígito	Decimal	Valor
2	3	3	768
1	A	10	160
0	B	11	11
		Total	939

Exemplo 2: Convertendo $(FF)_{16}$ para Decimal

$$\begin{aligned} FF_{16} &= F \times 16^1 + F \times 16^0 \\ &= 15 \times 16 + 15 \times 1 \\ &= 240 + 15 \\ &= 255_{10} \end{aligned}$$

Curiosidade: $(FF)_{16}$ é equivalente a $(11111111)_2$ e representa 255 em decimal.

Exemplo 3: Convertendo $(1000)_{16}$ para Decimal

$$\begin{aligned} 1000_{16} &= 1 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 \\ &= 1 \times 4096 + 0 + 0 + 0 \\ &= 4096_{10} \end{aligned}$$

1.4. Comparação entre as Bases

Tabela Comparativa

Decimal	Binário	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	10000	10
32	100000	20
64	1000000	40
128	10000000	80
255	11111111	FF
256	100000000	100

Potências Importantes

Potências de 2 (Binário)

Base 2	Decimal	Base 2	Decimal
2^0	1	2^6	64
2^1	2	2^7	128
2^2	4	2^8	256
2^3	8	2^9	512
2^4	16	2^{10}	1.024
2^5	32	2^{11}	2.048

Potências de 16 (Hexadecimal):

Hexadecimal	Decimal
16^0	1
16^1	16
16^2	256
16^3	4.096

1.5. Conversões Passo a Passo

1.5.1. Decimal → Binário

Método: Divisões Sucessivas por 2

Passos:

- i. Divida o número decimal por 2
- ii. Anote o **resto** (0 ou 1)
- iii. Divida o **quociente** por 2
- iv. Repita até que o quociente seja 0
- v. Leia os restos **de baixo para cima**

Exemplo 1: $(45)_{10} \rightarrow$ Binário

$$45 \div 2 = 22 \text{ resto } 1$$

$$22 \div 2 = 11 \text{ resto } 0$$

$$11 \div 2 = 5 \text{ resto } 1$$

$$5 \div 2 = 2 \text{ resto } 1$$

$$2 \div 2 = 1 \text{ resto } 0$$

$$1 \div 2 = 0 \text{ resto } 1$$

Resultado: $(101101)_2 \leftarrow$ Leia de baixo para cima

Verificação:

$$101101_2 = 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$$

$$101101_2 = 32 + 8 + 4 + 1$$

$$101101_2 = 45_{10}$$

Exemplo 2: $(128)_{10} \rightarrow$ Binário

$$128 \div 2 = 64 \text{ resto } 0$$

$$64 \div 2 = 32 \text{ resto } 0$$

$$32 \div 2 = 16 \text{ resto } 0$$

$$16 \div 2 = 8 \text{ resto } 0$$

$$8 \div 2 = 4 \text{ resto } 0$$

$$4 \div 2 = 2 \text{ resto } 0$$

$$2 \div 2 = 1 \text{ resto } 0$$

$$1 \div 2 = 0 \text{ resto } 1$$

Resultado: $(10000000)_2$

1.5.2. Binário → Decimal

Método: Soma das Potências de 2

Passos:

- i. Multiplique cada bit pela sua potência de 2
- ii. Some todos os resultados

Exemplo 1: $(11001)_2 \rightarrow$ Decimal

$$11001_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$11001_2 = 1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1$$

$$11001_2 = 16 + 8 + 1$$

$$11001_2 = 25_{10}$$

Representação Visual:

Posição	4	3	2	1	0
Bit	1	1	0	0	1
Valor	16	8	0	0	1
Total	25				

Exemplo 2: $(10101010)_2 \rightarrow \text{Decimal}$

$$\begin{aligned}
 10101010_2 &= 1 \times 128 + 0 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 \\
 &= 128 + 32 + 8 + 2 \\
 &= 170_{10}
 \end{aligned}$$

1.5.3. Decimal $\rightarrow$ Hexadecimal

Método: Divisões Sucessivas por 16

Passos:

- Divida o número decimal por 16
- Anote o **resto** (convertendo para A-F se > 9)
- Divida o **quociente** por 16
- Repita até que o quociente seja 0
- Leia os restos **de baixo para cima**

Exemplo 1: $(315)_{10} \rightarrow \text{Hexadecimal}$

$$\begin{aligned}
 315 \div 16 &= 19 \text{ resto } 11 (B) \\
 19 \div 16 &= 1 \text{ resto } 3 \\
 1 \div 16 &= 0 \text{ resto } 1
 \end{aligned}$$

Lendo de baixo para cima: $(13B)_{16}$

Verificação:

$$\begin{aligned}
 13B_{16} &= 1 \times 256 + 3 \times 16 + 11 \times 1 \\
 13B_{16} &= 256 + 48 + 11 \\
 13B_{16} &= 315_{10}
 \end{aligned}$$

Exemplo 2: $(4096)_{10} \rightarrow \text{Hexadecimal}$

$$\begin{aligned}
 4096 \div 16 &= 256 \text{ resto } 0 \\
 256 \div 16 &= 16 \text{ resto } 0 \\
 16 \div 16 &= 1 \text{ resto } 0 \\
 1 \div 16 &= 0 \text{ resto } 1
 \end{aligned}$$

Resultado: $(1000)_{16}$

1.5.4. Hexadecimal $\rightarrow$ Decimal

Método: Soma das Potências de 16

Passos:

- Multiplique cada dígito pela sua potência de 16
- Converta A-F para seus valores decimais (10-15)
- Some todos os resultados

Exemplo 1: $(2F7)_{16} \rightarrow \text{Decimal}$

$$\begin{aligned} 2F7_{16} &= 2 \times 16^2 + F \times 16^1 + 7 \times 16^0 \\ 2F7_{16} &= 2 \times 256 + 15 \times 16 + 7 \times 1 \\ 2F7_{16} &= 512 + 240 + 7 \\ 2F7_{16} &= 759_{10} \end{aligned}$$

Exemplo 2: $(ABC)_{16} \rightarrow \text{Decimal}$

$$\begin{aligned} ABC_{16} &= A \times 16^2 + B \times 16^1 + C \times 16^0 \\ ABC_{16} &= 10 \times 256 + 11 \times 16 + 12 \times 1 \\ ABC_{16} &= 2560 + 176 + 12 \\ ABC_{16} &= 2748_{10} \end{aligned}$$

1.5.5. Binário $\rightarrow$ Hexadecimal

Método: Agrupamento de 4 Bits

Passos:

- Agrupe os bits de **direita para esquerda em grupos de 4**
- Converta cada grupo para seu equivalente em hexadecimal
- Combine os resultados

Tabela de Referência Rápida

Binário	Hexadecimal	Binário	Hexadecimal
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

Exemplo 1: $(11010110)_2 \rightarrow \text{Hexadecimal}$

$$\begin{aligned} \text{Agrupando: } &1101 \mid 0110 \\ &1101 \rightarrow D \\ &0110 \rightarrow 6 \end{aligned}$$

Resultado: $(D6)_{16}$

Verificação:

$$\begin{aligned} D6_{16} &= 13 \times 16 + 6 \times 1 = 208 + 6 = 214_{10} \\ 11010110_2 &= 128 + 64 + 16 + 4 + 2 = 214_{10} \end{aligned}$$

Exemplo 2: $(101111001010)_2 \rightarrow \text{Hexadecimal}$

$$\begin{aligned} \text{Agrupando: } &1011 \mid 1100 \mid 1010 \\ &1011 \rightarrow B \\ &1100 \rightarrow C \\ &1010 \rightarrow A \end{aligned}$$

Resultado: $(BCA)_{16}$

1.5.6. Hexadecimal → Binário

Método: Expandir cada Dígito em 4 Bits

Passos:

- i. Converta cada dígito hexadecimal em seu equivalente de 4 bits
- ii. Concatene os bits
- iii. Simplifique removendo zeros à esquerda (opcional)

Exemplo 1: $(5E)_{16} \rightarrow$ Binário

$5 \rightarrow 0101$

$E \rightarrow 1110$

Resultado: $(01011110)_2$ ou $(1011110)_2$

Verificação:

$$01011110_2 = 0 + 64 + 16 + 8 + 4 + 2 + 0 = 94_{10}$$

$$5E_{16} = 5 \times 16 + 14 \times 1 = 80 + 14 = 94_{10}$$

Exemplo 2: $(A3F)_{16} \rightarrow$ Binário

$A \rightarrow 1010$

$3 \rightarrow 0011$

$F \rightarrow 1111$

Resultado: $(101000111111)_2$

1.6. Exercícios Propostos

- a) Converta $(18)_{10}$ para binário
- b) Converta $(1010)_2$ para decimal
- c) Converta $(11111111)_2$ para hexadecimal
- d) Converta $(157)_{10}$ para binário
- e) Converta $(3C5)_{16}$ para decimal
- f) Converta $(101101101)_2$ para decimal
- g) Converta $(1023)_{10}$ para binário e hexadecimal
- h) Converta $(ABCDEF)_{16}$ para decimal
- i) Converta $(111110000111)_2$ para hexadecimal e decimal

2. Representação de Números em Computadores

Classificação Geral dos Tipos

- i. INTEIROS (números sem decimal)
 - a) Sem sinal (unsigned) - apenas positivos
 - b) Com sinal (signed) - positivos e negativos
- ii. PONTO FLUTUANTE (números com decimal)
 - a) Float (precisão simples)
 - b) Double (precisão dupla)

2.1. Representação de Números Inteiros

2.1.1. Números Inteiros Sem Sinal (*Unsigned*)

Características:

- Apenas **números positivos** (≥ 0)
- Todos os bits representam a **magnitude**
- **Sem bit de sinal**

Exemplo: Inteiro de 8 bits Sem Sinal

00000000 = 0
00000001 = 1
00000010 = 2
...
11111111 = 255

Intervalo: 0 até 255 ($2^8 - 1$)

Fórmula Geral

Para um inteiro sem sinal de **n bits**:

$$\text{Intervalo} = 0 \text{ até } 2^n - 1$$

Exemplos de Intervalos

Tipo	Bits	Intervalo
uint8	8	0 até 255
uint16	16	0 até 65.535
uint32	32	0 até 4.294.967.295
uint64	64	0 até 18.446.744.073.709.551.615

2.1.2. Números Inteiros Com Sinal (Signed)

Características:

- Números **positivos e negativos**
- **Primeiro bit** é o bit de sinal (0 = positivo, 1 = negativo)
- Usa **complemento de 2** para representar negativos

Fórmula Geral

Para um inteiro com sinal de **n bits**:

$$\text{Intervalo} = -2^{\{n-1\}} \text{ até } 2^{\{n-1\}} - 1$$

Exemplos de Intervalos

Tipo	Bits	Intervalo
int8	8	-128 até 127
int16	16	-32.768 até 32.767
int32	32	-2.147.483.648 até 2.147.483.647
int64	64	-9.223.372.036.854.775.808 até 9.223.372.036.854.775.807

2.2. Tipos Inteiros no Octave

2.2.1. int8 - Inteiro de 8 bits com Sinal

Características

- Tamanho: 8 bits
- Intervalo: -128 até 127
- Uso: Dados muito pequenos, economizar memória

Exemplo 1: Criando int8 no Octave

```
>> a = int8(42)
a = 42

>> b = int8(-50)
b = -50

>> c = int8(200)      % Valor fora do intervalo!
c = 127               % Saturação (overflow)
```

Exemplo 2: Representação Binária

+42 em int8: 00101010
-50 em int8: 11001110 (complemento de 2)
+127 em int8: 01111111 (máximo)
-128 em int8: 10000000 (mínimo)

Exemplo 3: Operações com int8

```
>> a = int8(100);
>> b = int8(50);
>> c = a + b
c = 127          % 100 + 50 = 150, mas saturou em 127

>> d = int8(-100) + int8(-50)
d = -128        % -100 - 50 = -150, mas saturou em -128
```

2.2.2. int16 - Inteiro de 16 bits com Sinal

Características

- Tamanho: 16 bits (2 bytes)
- Intervalo: -32.768 até 32.767
- Uso: Números médios, aplicações de áudio

Exemplo: Criando int16

```
>> a = int16(10000)
a = 10000

>> b = int16(-20000)
b = -20000

>> c = int16(50000)    % Fora do intervalo
c = 32767              % Saturação
```

2.2.3. int32 - Inteiro de 32 bits com Sinal

Características

- Tamanho: 32 bits (4 bytes)
- Intervalo: -2.147.483.648 até 2.147.483.647
- Uso: Inteiros padrão, programação geral

Exemplo: Criando int32

```
>> a = int32(1000000)
a = 1000000

>> b = int32(-1000000)
b = -1000000

>> c = int32(3000000000)    % Fora do intervalo
c = 2147483647              % Saturação
```

2.2.4. int64 - Inteiro de 64 bits com Sinal

Características

- Tamanho: 64 bits (8 bytes)
- Intervalo: -9.223.372.036.854.775.808 até 9.223.372.036.854.775.807
- Uso: Números muito grandes, timestamps, IDs

Exemplo: Criando int64

```
>> a = int64(9000000000)
a = 9000000000

>> b = int64(-9000000000)
b = -9000000000
```

2.2.5. Comando whos para Verificar Tipos

```
>> a = int8(42);
>> b = int16(1000);
>> c = int32(100000);
>> whos
Name      Size      Bytes  Class      Attributes
a         1x1         1      int8
b         1x1         2      int16
c         1x1         4      int32
```

2.3. Tabela Comparativa de Tipos Inteiros

Tipo	Bits	Bytes	Mín	Máx	Uso
int8	8	1	-128	127	Dados pequenos
int16	16	2	-32.768	32.767	Áudio
int32	32	4	-2.1 bi	2.1 bi	Padrão
int64	64	8	-9.2e18	9.2e18	Grandes números

2.4. Representação de Números Negativos

Em um sistema de 8 bits, como representar -5? Não podemos simplesmente usar um sinal (+ ou -) como fazemos na matemática.

2.4.1. Complemento de 2 - O Padrão Moderno

- Operações aritméticas funcionam normalmente
- Subtração = Adição de negativos
- Apenas uma representação de zero
- Hardware é simples

2.4.2. Como Converter um Número para Seu Negativo em Complemento de 2

Passo a Passo:

- Represente o número positivo em binário**
- Inverta todos os bits** (0→1, 1→0) - Complemento de 1
- Adicione 1 ao resultado** - Complemento de 2

Exemplo 1: Converter +5 em -5 (8 bits)

- a) Passo 1: +5 em binário (8 bits)

00000101

- b) Passo 2: Inverter todos os bits (Complemento de 1)

11111010

- c) Passo 3: Adicionar 1

$$\begin{array}{r} 11111010 \\ + \quad 1 \\ \hline 11111011 \end{array}$$

- d) Resultado: -5 em C2 = 11111011

Verificação:

11111011 com bit de sinal:

Bit de sinal = 1 (negativo)

$$\begin{aligned} \text{Magnitude} &= -2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^1 + 2^0 \\ &= -128 + 64 + 32 + 16 + 8 + 2 + 1 \\ &= -128 + 123 \\ &= -5 \end{aligned}$$

Exemplo 2: Converter +42 em -42 (8 bits)

1. Passo 1: +42 em binário (8 bits)
00101010
2. Passo 2: Inverter todos os bits
11010101
3. Passo 3: Adicionar 1
$$\begin{array}{r} 11010101 \\ + \quad 1 \\ \hline 11010110 \end{array}$$
4. Resultado: -42 em C2 = 11010110

Verificação:

$$\begin{aligned} &11010110 \text{ com interpretação em C2:} \\ &= -128 + 64 + 16 + 4 + 2 \\ &= -128 + 86 \\ &= -42 \end{aligned}$$

2.4.5. Verificação em Octave

```
>> a = int8(-5)
a = -5

% Vendo a representação binária
>> fprintf('%s\n', dec2bin(typecast(a, 'uint8')))
11111011

% Confermando que é -5:
>> double(a)
ans = -5

% Verificando +5:
>> fprintf('%s\n', dec2bin(typecast(int8(5), 'uint8')))
00000101
```

2.5. Representação de Números em Ponto Flutuante

Como representar números como 3.14159 ou 0.00001 em binário? Usar notação **científica binária**

$$\text{Número} = (-1)^{\{sinal\}} \times mantissa \times 2^{\{expoente\}}$$

2.5.1. Padrão IEEE 754

O padrão moderno para ponto flutuante é o **IEEE 754**.

Estrutura Geral

[Bit de Sinal] [Expoente] [Mantissa]

1 bit n bits m bits

2.5.2. Float (Precisão Simples - 32 bits)

Estrutura

[Sinal: 1 bit] [Expoente: 8 bits] [Mantissa: 23 bits]

0 ou 1 0-255 23 bits de precisão

Características

- Tamanho: 32 bits (4 bytes)
- Precisão: ~7 dígitos decimais
- Intervalo: $\pm 1.4 \times 10^{-45}$ até $\pm 3.4 \times 10^{38}$

Exemplo: Criando Float no Octave

```
>> a = single(3.14159)
a = 3.1416

>> b = single(-123.456)
b = -123.4560

>> c = single(0.00001)
c = 1.0000e-05

% Verificando tamanho
>> whos a
Name      Size      Bytes Class  Attributes
a         1x1         4 single
```

2.5.3. Double (Precisão Dupla - 64 bits)

Estrutura

[Sinal: 1 bit] [Expoente: 11 bits] [Mantissa: 52 bits]

0 ou 1 0-2047 52 bits de precisão

Características

- Tamanho: 64 bits (8 bytes)
- Precisão: ~15 dígitos decimais
- Intervalo: $\pm 2.2 \times 10^{-308}$ até $\pm 1.8 \times 10^{308}$

Exemplo: Criando Double no Octave

```
>> a = 3.14159265358979
a = 3.1416 % Mostra menos, mas armazena mais

>> b = -123.456789012345
b = -123.4568

% Double é o PADRÃO no Octave!
>> whos a
Name      Size      Bytes Class  Attributes
a         1x1         8 double
```

2.5.4. Comparação Float vs Double

```
>> f = single(1/3)
f = 0.3333

>> d = double(1/3)
d = 0.3333

>> f - d
ans = 1.4901e-08

% Double tem MUITO mais precisão!
```

2.6. Soma Binária

7.1 Regras Básicas de Soma Binária

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10$ (0 com "vai 1")
- $1 + 1 + 1$ (carry) = 11 (1 com "vai 1")

Exemplo 1: Soma Simples ($5 + 3$)

$$\begin{array}{r} 00000101 \text{ (+5)} \\ + 00000011 \text{ (+3)} \\ \hline 00001000 \text{ (+8)} \end{array}$$

Verificação: $5 + 3 = 8$

Passo a Passo:

- Posição 0: $1 + 1 = 0$ (vai 1)
- Posição 1: $0 + 1 + 1(\text{carry}) = 0$ (vai 1)
- Posição 2: $1 + 0 + 1(\text{carry}) = 0$ (vai 1)
- Posição 3: $0 + 0 + 1(\text{carry}) = 1$ (vai 0)
- Posição 4 a 7: 0

Exemplo 2: Soma com Números Maiores ($42 + 25$)

$$\begin{array}{r} 00101010 \text{ (+42)} \\ + 00011001 \text{ (+25)} \\ \hline 01000011 \text{ (+67)} \end{array}$$

Verificação: $42 + 25 = 67$

Detalhado:

- 00101010
- + 00011001
- _____
- Position 0: $0+1=1$
- Position 1: $1+0=1$
- Position 2: $0+0=0$ (com carry 1 da posição anterior? Não)

- vii. Position 3: $1+1=0$ (vai 1)
- viii. Position 4: $0+1+1=0$ (vai 1)
- ix. Position 5: $1+0+1=0$ (vai 1)
- x. Position 6: $0+0+1=1$
- xi. Result: 01000011 = 67

Exemplo 3: Soma com Números Negativos (-5 + 3)

Usando complemento de 2 (8 bits):

$$\begin{aligned}-5 &= 11111011 \\ +3 &= 00000011\end{aligned}$$

$$\begin{array}{r}11111011 \text{ } (-5) \\ + 00000011 \text{ } (+3) \\ \hline 11111110 \text{ } (-2)\end{array}$$

Verificação: $-5 + 3 = -2$

Observação Importante: A soma funciona **normalmente** mesmo com números negativos em complemento de 2!

Exemplo 4: Soma de Dois Negativos (-10 + -20)

$$\begin{aligned}-10 &= 11110110 \text{ (complemento de 2)} \\ -20 &= 11101100 \text{ (complemento de 2)}\end{aligned}$$

$$\begin{array}{r}11110110 \text{ } (-10) \\ + 11101100 \text{ } (-20) \\ \hline 111100010 \text{ } (-30) \\ \text{(descarte o carry final)} \\ = 11100010 \text{ } (-30)\end{array}$$

Verificação: $-10 + (-20) = -30$

Overflow em Soma

- Quando o resultado não cabe no número de bits disponível!

Exemplo: Overflow com 8 bits

$$\begin{aligned}+100 &= 01100100 \\ +50 &= 00110010\end{aligned}$$

$$\begin{array}{r}01100100 \text{ } (+100) \\ + 00110010 \text{ } (+50) \\ \hline\end{array}$$

10010110 (resultado errado!)

- Interpretação incorreta: O bit de sinal é 1 → NEGATIVO!
- Resultado mostrado: -106 (ERRADO!)
- Resultado correto: +150 (não cabe em 8 bits)

2.7. Exercícios Propostos

- a) Qual é o intervalo de valores para um **int8**?
- b) Qual é o intervalo de valores para um **int32**?
- c) Represente o número **+10** em binário (8 bits)
- d) Represente o número **-10** em complemento de 2 (8 bits)
- e) Converta **11111111** (complemento de 2) para decimal
- f) Some em binário: **00000101 + 00000011**
- g) Qual é a diferença entre **float** e **double**?
- h) Represente **-50** em complemento de 2 (8 bits) e mostre que **-(-50) = +50**
- i) Some em binário: **01100100 + 00110010** (mostrando overflow com 8 bits)
- j) Represente **-128 em complemento de 2** (8 bits). Por que **-(-128)** não funciona?
- k) Crie um script que converta um número decimal para binário, depois para complemento de 2 (se negativo)

3. Erros Numéricos em Cálculo Numérico

Erros numéricos são diferenças entre o valor exato (teórico) e o valor calculado/armazenado por um computador.

Razões principais:

- a) LIMITAÇÕES DE REPRESENTAÇÃO
 - Computadores usam número finito de bits
 - Não conseguem representar todos os reais
- b) OPERAÇÕES EM PONTO FLUTUANTE
 - Arredondamentos necessários
 - Cancelamento catastrófico possível
- c) MÉTODOS NUMÉRICOS
 - Aproximações (séries, iterações)
 - Discretização de problemas contínuos
- d) DADOS DE ENTRADA
 - Medições imprecisas
 - Valores arredondados

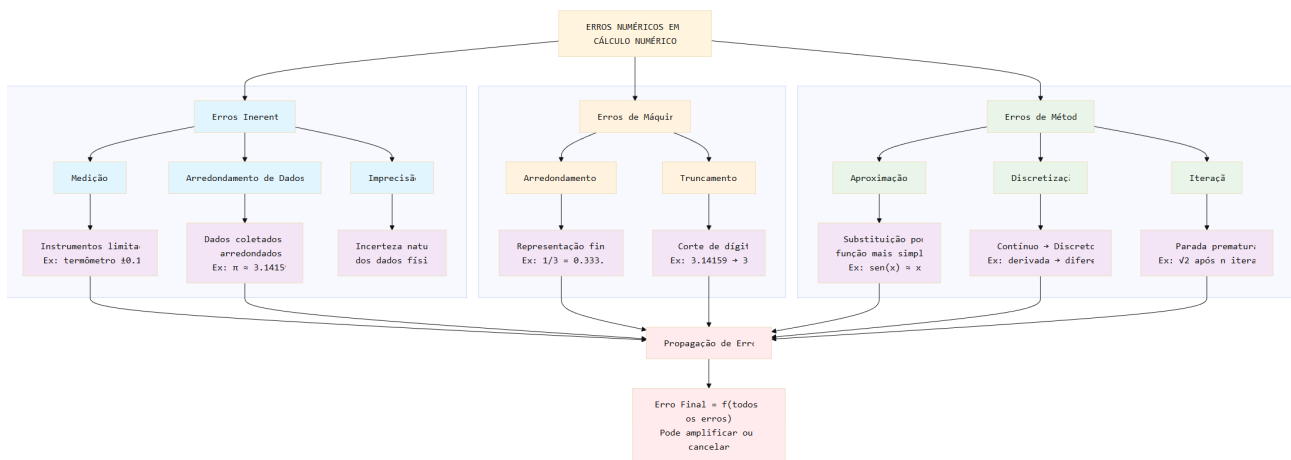
Importância de Estudar Erros

- Entender confiabilidade dos resultados
- Melhorar precisão de cálculos
- Detectar problemas em computações
- Escolher métodos apropriados
- Validar resultados numéricos

3.1. Classificação de Erros

- ERROS NUMÉRICOS
 - ERROS INERENTES (Dados de entrada)
 - Erros de medição
 - Erros de arredondamento de dados
 - Dados incompletos ou imprecisos
- ERROS DE MÁQUINA (Representação)
 - Erro de arredondamento
 - Erro de truncamento
- ERROS DE MÉTODO (Algoritmo)
 - Erro de aproximação
 - Erro de discretização
 - Erro de iteração (convergência)

3.2. Visualização dos Erros no Processo Computacional



3.3. Erro Absoluto e Erro Relativo

3.3.1. Erro Absoluto

Definição

$$E_a = x_{\text{exato}} - x_{\text{aproximado}}$$

Onde:

- x_{exato} = valor verdadeiro (teórico)
- $x_{\text{aproximado}}$ = valor calculado/armazenado

Características

- Tem as mesmas unidades do valor original
- Sempre não-negativo
- Fácil de calcular
- Depende da magnitude do valor

Exemplo 1: Calculando Erro Absoluto

Valor exato: $\pi = 3.14159265\dots$

Valor aproximado: 3.14159 (5 casas decimais)

$$E_a = |3.14159265\dots - 3.14159| = |0.00000265\dots| = 0.00000265\dots$$

3.3.2. Erro Relativo

Definição

$$E_r = \frac{|x_{\text{exato}} - x_{\text{aproximado}}|}{|x_{\text{exato}}|} = \frac{E_a}{|x_{\text{exato}}|}$$

Ou em Percentual

$$E_r(\%) = E_r \times 100\%$$

Características

- Adimensional (sem unidades)
- Mais significativo que erro absoluto
- Permite comparações entre quantidades diferentes
- Melhor indicador de precisão

Exemplo 2: Calculando Erro Relativo

Caso A: Aproximando π

Valor exato: $\pi = 3.14159265\dots$

Valor aproximado: 3.14159

$$E_a = 0.00000265\dots$$

$$E_r = 0.00000265\dots / 3.14159265\dots \approx 0.000000844 = 0.0000844\%$$

Caso B: Aproximando 0.001

Valor exato: 0.001

Valor aproximado: 0.0009

$$E_a = |0.001 - 0.0009| = 0.0001$$

$$E_r = 0.0001 / 0.001 = 0.1 = 10\%$$

3.3.3. Comparação Erro Absoluto vs Relativo

Tabela Comparativa

Grandeza	Valor Exato	Valor Aprox.	E_a	E_r	$E_r(\%)$
Distância	1000 km	999 km	1 km	0.001	0.1%
Altura	1.80 m	1.79 m	0.01 m	0.0056	0.56%
Massa	0.001 kg	0.0009 kg	0.0001 kg	0.1	10%

Conclusão: O erro relativo é mais informativo.

3.3.4. Erro Relativo Percentual Aproximado

Quando Não Conhecemos o Valor Exato

Às vezes, não sabemos o valor exato. Usamos uma aproximação anterior como referência:

$$E_{r,aprox(\%)} = \left| \frac{\{x_n - x_{\{n-1\}}\}}{\{x_n\}} \right| \times 100\%$$

Onde:

- $x_{\{n\}}$ = aproximação atual
- $x_{\{n-1\}}$ = aproximação anterior

Exemplo: Método Iterativo

- Iteração 1: $x_1 = 3.1$
- Iteração 2: $x_2 = 3.14$
- Iteração 3: $x_3 = 3.1416$
- Iteração 4: $x_4 = 3.14159$

$$E_{r(\text{aprox})} \text{ de } x^2 = \frac{|3.14 - 3.1|}{3.14} \times 100\% = 1.27\%$$

$$E_{r(\text{aprox})} \text{ de } x^3 = \frac{|3.1416 - 3.14|}{3.1416} \times 100\% = 0.051\%$$

$$E_{r(\text{aprox})} \text{ de } x^4 = \frac{|3.14159 - 3.1416|}{3.14159} \times 100\% = 0.0029\%$$

Observação: O erro relativo diminui a cada iteração

3.4. Erro de Truncamento

Erro de truncamento ocorre quando uma representação **infinita** é substituída por uma **finita**.

SÉRIE INFINITA

↓

SÉRIE TRUNCADA (n termos)

↓

ERRO = Resto da série não calculado

Exemplo 1: Série de Maclaurin para e^x

$$f(x) = \sum_{n=0}^{\infty} \frac{f^n(0) x^n}{n!} = f(0) + \frac{f'(0)}{1!}x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \dots$$

- **Série Completa (Infinita)**

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

- **Série Truncada (Primeiros 4 Termos)**

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!}$$

- **Considere $x = 1$, e^1 (Número de Euler)**

- Valor exato: $e^1 \approx 2.71828182845904\dots$
- Usando 1 termo: $e \approx 1$; $E_r \approx 63.2\%$
- Usando 2 termos: $e \approx 1 + 1 = 2$; $E_r \approx 26.4\%$

- Usando 3 termos: $e \approx 2 + 0.5 = 2.5$; $E_r \approx 8.0\%$
- Usando 4 termos: $e \approx 2.5 + 0.167 = 2.667$; $E_r \approx 1.84\%$
- Usando 5 termos: $e \approx 2.667 + 0.042 = 2.708$; $E_r \approx 0.368\%$

Exemplo 2: Série de Taylor para $\sin(x)$

- **Série Completa**

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

- **Exemplo Numérico:** $\sin(\pi/6) = 0.5$
 - Valor exato: $\sin(\pi/6) = 0.5$
 - Usando 1 termo: $\sin\left(\frac{\pi}{6}\right) \approx \frac{\pi}{6} \approx 0.5236$; $E_a \approx 0.0236$
 - Usando 2 termos: $\sin\left(\frac{\pi}{6}\right) \approx 0.5236 - 0.0451 \approx 0.4785$; $E_a \approx 0.0215$
 - Usando 3 termos: $\sin\left(\frac{\pi}{6}\right) \approx 0.4785 + 0.0027 \approx 0.4812$; $E_a \approx 0.0188$
 - Usando 4 termos: $\sin\left(\frac{\pi}{6}\right) \approx 0.4812 - 0.00007 \approx 0.4999$; $E_a \approx 0.0001$

3.5. Erro de Arredondamento

Erro de arredondamento ocorre quando um número é representado com um número limitado de dígitos.

5.2 Causas do Erro de Arredondamento

- PONTO FLUTUANTE FINITO
 - Mantissa tem número limitado de bits
 - Não consegue representar todos os reais
- REPRESENTAÇÃO BINÁRIA
 - Alguns decimais não têm representação exata
 - Exemplo: 0.1 em decimal = 0.0001100110011... em binário
- ARREDONDAMENTO DE OPERAÇÕES
 - Após cada operação, resultado é arredondado
 - Acumula ao longo de cálculos longos

Exemplo 1: Representação de Decimais em Binário

Decimal 0.1 em Binário:

Processo: Para converter a parte fracionária de um número decimal para binário, multiplica-se repetidamente a parte fracionária por 2, anotando a parte inteira resultante a cada passo. O processo continua até que a parte fracionária se torne zero (o que não ocorre para 0.1) ou até atingir a precisão desejada.

$0.1 \times 2 = 0.2$ (bit 0)
 $0.2 \times 2 = 0.4$ (bit 0)
 $0.4 \times 2 = 0.8$ (bit 0)
 $0.8 \times 2 = 1.6$ (bit 1)
 $0.6 \times 2 = 1.2$ (bit 1)
 $0.2 \times 2 = 0.4$ (bit 0) ← REPETE!

$0.1_{10} = 0.0001100110011..._2$ (PERIÓDICO!)

- Em 32 bits (float): 0.10000000149011611...
- Em 64 bits (double): 0.100000000000000001...

Exemplo 2: Acúmulo de Erro de Arredondamento

```
% Demonstração em Octave

% Método 1: Usando 0.1 direto
>> soma = 0;
>> for i = 1:10
>>     soma = soma + 0.1;
>> end
>> soma
ans = 1.00000000000000009      % Deveria ser 1.0!

% Erro acumulado
>> erro = soma - 1
erro = 8.8817841970012523e-16

% Método 2: Usando fração
>> soma2 = 10 * 0.1
soma2 = 1
```

3.5.1. Regras de Arredondamento

Arredondamento Padrão (Round-to-nearest)

Se o dígito a cortar é:

- < 5 → arredondar para baixo
- $= 5$ → arredondar para o par mais próximo
- > 5 → arredondar para cima

Exemplos:

- $1.234 \rightarrow 1.23$ ($4 < 5$)
- $1.235 \rightarrow 1.24$ (5, arredonda para par)
- $1.236 \rightarrow 1.24$ ($6 > 5$)

Arredondamento por Truncamento

Simplesmente remover os dígitos finais

Exemplos:

- $1.234 \rightarrow 1.23$
- $1.236 \rightarrow 1.23$
- $1.239 \rightarrow 1.23$

3.6. Erro de Cancelamento Catastrófico

Cancelamento catastrófico ocorre quando subtraímos dois números **muito próximos**, causando perda significativa de dígitos significativos.

Quando subtraímos $x - y$ com $x \approx y$:

$$x = 0.123456789...$$

$$y = 0.123456780\dots$$

$$x - y = 0.000000009 \text{ (apenas 1 dígito significativo)}$$

Perdemos os primeiros 7 dígitos significativos!

Exemplo: Diferenças Pequenas

- Calcular: $\frac{(\sqrt{1+x}-1)}{x}$ para $x = 1e^{-12}$
- Valor exato: Aproximadamente 0.5
- Método tradicional:

$$\sqrt{1 + 1e^{-12}} \approx 1.00000000005$$

$$\text{Numerador} = 1.00000000005 - 1 = 0.00000000005 \text{ (cancelamento!)}$$

$$\text{Resultado} = \frac{0.00000000005}{1e^{-12}} = 0.5 \text{ (Por sorte!)}$$

- Mas com arredondamento:

$$\sqrt{1 + 1e^{-12}} \text{ pode estar entre } 1 \text{ ou } 1.00000000001$$

- Resultado pode estar errado!
-

3.6.1. Identificando Cancelamento Catastrófico

- Resultado com muitos zeros após vírgula
- Resultado com apenas 1-2 dígitos significativos
- Diferença entre números muito próximos
- Instabilidade numérica

3.7. Propagação de Erros

Propagação de erros estuda como erros em dados de entrada se refletem no resultado final.

- **Adição:** $z = x + y$

Se Δx e Δy são erros de entrada:

$$\Delta z = \Delta x + \Delta y$$

$$E_{r,z} = \frac{\Delta x + \Delta y}{x + y}$$

Exemplo

Considerando $x = 10 \pm 0.1$ e $y = 20 \pm 0.2$

$$z = x + y = 30$$

$$\Delta z = 0.1 + 0.2 = 0.3$$

$$E_{r,z} = 0.3/30 \approx 1\%$$

- **Subtração:** $z = x - y$

$$\Delta z = \Delta x + \Delta y$$

$$E_{r,z} = \frac{\Delta x + \Delta y}{x - y}$$

Atenção: Se $x \approx y$ o erro relativo pode ser **MUITO GRANDE!**

Exemplo (Cancelamento!)

Considerando $x = 10.1 \pm 0.1$ e $y = 10.0 \pm 0.1$

$$z = x - y = 0.1$$

$$\Delta z = 0.1 + 0.1 = 0.2$$

$$E_{r,z} = \frac{0.2}{0.1} = 200\% \text{ (DESASTRE!)}$$

- **Multiplicação:** $z = x \cdot y$

$$\frac{\Delta z}{z} = \frac{\Delta x}{x} + \frac{\Delta y}{y}$$

(Soma dos erros relativos)

Exemplo

Considerando $x = 10 \pm 0.1$ ($E_{r,x} = 1\%$) e $y = 20 \pm 0.2$ ($E_{r,y} = 1\%$)

$$z = 200$$

$$E_{r,z} \approx 1\% + 1\% = 2\%$$

$$\Delta z \approx 200 \times 0.02 = 4$$

- **Divisão:** $z = x/y$

$$\frac{\Delta z}{z} = \frac{\Delta x}{x} + \frac{\Delta y}{y}$$

(Mesma que multiplicação)

- **Potência:** $z = x^n$

$$\frac{\Delta z}{z} = n \cdot \frac{\Delta x}{x}$$

Exemplo

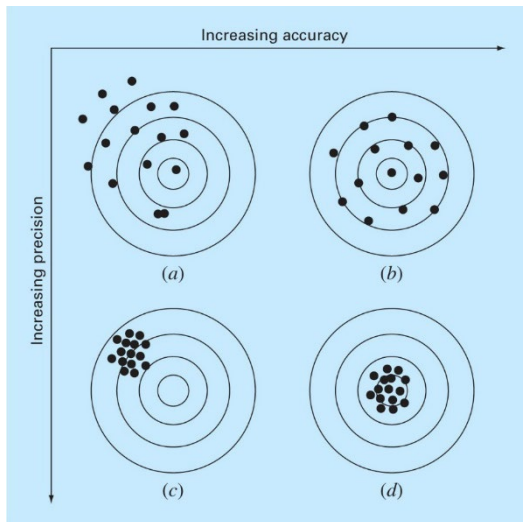
$$x = 2 \pm 0.01 \text{ } (E_{r,x} = 0.5\%)$$

$$z = x^3$$

$$E_{r,z} \approx 3 \times 0.5\% = 1.5\%$$

3.8. Acurácia vs Precisão

- **Acurácia (Accuracy)**
 - Grau de PROXIMIDADE com o valor VERDADEIRO
 - Avalia sistemático do método
 - Relacionado a erros sistemáticos
 - Pode ser melhorado com melhor algoritmo
 - Afetado por erros de truncamento
- **Precisão (Precision)**
 - Grau de REPETIBILIDADE de medições/cálculos
 - Avalia dispersão dos resultados
 - Relacionado a erros aleatórios
 - Afetado por arredondamento
 - Número de dígitos significativos



- (a) Inacurado e impreciso
- (b) Acurado e impreciso
- (c) Inacurado e preciso
- (d) Acurado e preciso

Relação com Erros

- ACURÁCIA ← Controlada por:
 - Erro de truncamento (método)
 - Erro de método (discretização)
 - Precisão do algoritmo
 - Ordem de convergência
- PRECISÃO ← Controlada por:
 - Erro de arredondamento
 - Número de dígitos significativos
 - Tipo de dado (float vs double)
 - Acúmulo de erros de arredondamento

Exemplo Prático: Aproximação de π

- **Método 1:** $\pi \approx 22/7 = 3.142857...$
 - **Valor real:** $\pi = 3.141592...$
 - Acurácia: $E_r = 0.0407\%$ (baixa, converge para valor errado)
 - Precisão: 6 dígitos significativos
 - BAIXA ACURÁCIA, MÉDIA PRECISÃO

Tabela Comparativa Completa

Aspecto	Acurácia	Precisão
Definição	Proximidade com valor verdadeiro	Repetibilidade/Consistência
Relacionado a	Erros sistemáticos	Erros aleatórios
Número de casas	Não importa	Muito importante
Controlado por	Algoritmo/método	Arredondamento/representação
Melhorado por	Melhor algoritmo	Mais dígitos significativos
Símbolo	E_a , E_r	$\epsilon_{\text{máq}}$, dígitos
Foco	Valor correto	Variabilidade

3.9. Exemplos Práticos em Octave

3.9.1. Programa: Calculando Erros Absoluto e Relativo

```
% Script: calculo_erros.m

clear all; clc;

disp('=== CÁLCULO DE ERROS ABSOLUTO E RELATIVO ===');
disp(' ');

function [E_a, E_r, E_r_pct] = calcular_erros(x_exato, x_aprox)
    E_a = abs(x_exato - x_aprox);
    E_r = E_a / abs(x_exato);
    E_r_pct = E_r * 100;
end

% Exemplo 1: Aproximação de  $\pi$ 
disp('Exemplo 1: Aproximação de  $\pi$ ');
disp('_____');

pi_exato = pi; % Valor de referência do Octave
pi_aprox1 = 3.14;
pi_aprox2 = 3.14159;
pi_aprox3 = 3.141592653589793;

fprintf('Valor exato: %.15f\n\n', pi_exato);

aproximacoes = [pi_aprox1, pi_aprox2, pi_aprox3];
descricoes = {'3 casas (3.14)', '5 casas (3.14159)', '15 casas'};

for i = 1:length(aproximacoes)
    [E_a, E_r, E_r_pct] = calcular_erros(pi_exato, aproximacoes(i));
    fprintf('Aproximação: %s\n', descricoes{i});
    fprintf(' Valor: %.15f\n', aproximacoes(i));
    fprintf(' Erro absoluto: %.2e\n', E_a);
    fprintf(' Erro relativo: %.6f (%.6f%%)\n\n', E_r, E_r_pct);
end

% Exemplo 2: Número de Euler
disp(' ');
disp('Exemplo 2: Aproximação de e (Número de Euler)');
disp('_____');

e_exato = exp(1);
e_aprox1 = 2.7;
e_aprox2 = 2.718;
e_aprox3 = 2.718281828;

fprintf('Valor exato: %.15f\n\n', e_exato);

aproximacoes2 = [e_aprox1, e_aprox2, e_aprox3];
descricoes2 = {'2 casas (2.7)', '3 casas (2.718)', '9 casas'};

for i = 1:length(aproximacoes2)
    [E_a, E_r, E_r_pct] = calcular_erros(e_exato, aproximacoes2(i));
    fprintf('Aproximação: %s\n', descricoes2{i});
    fprintf(' Valor: %.15f\n', aproximacoes2(i));
    fprintf(' Erro absoluto: %.2e\n', E_a);
    fprintf(' Erro relativo: %.6f (%.6f%%)\n\n', E_r, E_r_pct);
end
```

Saída esperada:

=== CÁLCULO DE ERROS ABSOLUTO E RELATIVO ===

Exemplo 1: Aproximação de π

Valor exato: 3.141592653589793

Aproximação: 3 casas (3.14)

Valor: 3.1400000000000000

Erro absoluto: 1.59e-03

Erro relativo: 0.000507 (0.050747%)

Aproximação: 5 casas (3.14159)

Valor: 3.1415900000000000

Erro absoluto: 2.65e-06

Erro relativo: 0.000001 (0.000085%)

Aproximação: 15 casas

Valor: 3.141592653589793

Erro absoluto: 0.00e+00

Erro relativo: 0.000000 (0.000000%)

Exemplo 2: Aproximação de e (Número de Euler)

Valor exato: 2.718281828459045

Aproximação: 2 casas (2.7)

Valor: 2.7000000000000000

Erro absoluto: 1.83e-02

Erro relativo: 0.006705 (0.670548%)

Aproximação: 3 casas (2.718)

Valor: 2.7180000000000000

Erro absoluto: 2.82e-05

Erro relativo: 0.000010 (0.001036%)

Aproximação: 9 casas

Valor: 2.7182818280000000

Erro absoluto: 4.59e-10

Erro relativo: 0.000000 (0.000000%)

3.9.2. Programa: Erro de Truncamento em Série de Taylor

```
% Script: erro_truncamento.m

clear all; clc;

disp('=== ERRO DE TRUNCAMENTO EM SÉRIE DE TAYLOR ===');
disp(' ');

% Série de e^x
function resultado = exp_serie(x, n_termos)
    resultado = 0;
    for n = 0:(n_termos - 1)
        resultado = resultado + (x^n) / factorial(n);
    end
end

% Exemplo: e^1 (Número de Euler)
```

```

disp('Aproximação de e^1 usando série de Maclaurin');
disp('_____');

x = 1;
e_exato = exp(x);

fprintf('Valor exato: e^1 = %.15f\n\n', e_exato);
fprintf('n termos | Aproximação | Erro Absoluto | Erro Relativo\n');
fprintf('_____|_____|_____|_____\n');

for n = 1:10
    e_aprox = exp_serie(x, n);
    E_a = abs(e_exato - e_aprox);
    E_r = E_a / abs(e_exato);

    fprintf('%8d | %.14f | %.2e | %.2e\n', n, e_aprox, E_a, E_r);
end

% Série de sin(x)
disp(' ');
disp('Aproximação de sin(π/6) usando série de Taylor');
disp('_____');

function resultado = sin_serie(x, n_termos)
    resultado = 0;
    for n = 0:(n_termos - 1)
        resultado = resultado + ((-1)^n) * (x^(2*n + 1)) / factorial(2*n + 1);
    end
end

x = pi/6;
sin_exato = sin(x); % = 0.5

fprintf('Valor exato: sin(π/6) = %.15f\n\n', sin_exato);
fprintf('n termos | Aproximação | Erro Absoluto | Erro Relativo\n');
fprintf('_____|_____|_____|_____\n');

for n = 1:8
    sin_aprox = sin_serie(x, n);
    E_a = abs(sin_exato - sin_aprox);
    E_r = E_a / abs(sin_exato);

    fprintf('%8d | %.14f | %.2e | %.2e\n', n, sin_aprox, E_a, E_r);
end

```

Saída parcial:

=== ERRO DE TRUNCAMENTO EM SÉRIE DE TAYLOR ===

Aproximação de e^1 usando série de Maclaurin

Valor exato: e^1 = 2.718281828459045

n termos	Aproximação	Erro Absoluto	Erro Relativo
1	1.0000000000000000	1.72e+00	6.32e-01
2	2.0000000000000000	7.18e-01	2.64e-01
3	2.5000000000000000	2.18e-01	8.04e-02
4	2.6666666666666667	5.16e-02	1.90e-02
5	2.7083333333333333	9.95e-03	3.66e-03
6	2.7166666666666667	1.62e-03	5.95e-04
7	2.7180555555555556	2.27e-04	8.35e-05

3.9.3. Programa: Demonstrando Cancelamento Catastrófico

```
% Script: cancelamento_catastrofico.m
clear all; clc;

disp('=== CANCELAMENTO CATASTRÓFICO ===');
disp(' ');

% Problema:  $(\sqrt{1+x} - 1) / x$  para x pequeno

disp('Problema: Calcular  $(\sqrt{1+x} - 1) / x$  para x pequeno');
disp('_____');

function resultado = metodo_direto(x)
    resultado = (sqrt(1 + x) - 1) / x;
end

function resultado = metodo_racionalizado(x)
    resultado = 1 / (sqrt(1 + x) + 1);
end

% Valor teórico: quando  $x \rightarrow 0$ , resultado  $\rightarrow 1/2 = 0.5$ 

fprintf('Valor esperado: 0.5\n\n');
fprintf('x          | Direto          | Racionalizado   | Diferença\n');
fprintf('_____ | _____ | _____ | _____\n');

valores_x = [0.1, 1e-5, 1e-10, 1e-15, 1e-16, 1e-17];

for x = valores_x
    resultado_direto = metodo_direto(x);
    resultado_racional = metodo_racionalizado(x);
    diferenca = abs(resultado_direto - resultado_racional);

    fprintf('%0e      | %.14f | %.14f | %.2e\n', x, ...
            resultado_direto, resultado_racional, diferenca);
end

disp(' ');
disp('Observação: Método direto perde precisão para x muito pequeno!');
disp('          Método racionalizado permanece estável!');

% Exemplo 2: Raízes de equação quadrática
disp(' ');
disp('_____');
disp('Raízes de  $ax^2 + bx + c = 0$ ');
disp('_____');

disp(' ');
disp('Equação:  $x^2 + 1000x + 1 = 0$ ');

a = 1; b = 1000; c = 1;

% Método tradicional
disc = b^2 - 4*a*c;
x1_trad = (-b + sqrt(disc)) / (2*a); % Cancelamento!
x2_trad = (-b - sqrt(disc)) / (2*a); % Sem problema

% Método alternativo (evita cancelamento)
```

```
x1_alt = -2*c / (b + sqrt(disc));

fprintf('Fórmula tradicional:\n');
fprintf('  x1 = %.15f (CANCELAMENTO!)\n', x1_trad);
fprintf('  x2 = %.15f\n\n', x2_trad);

fprintf('Fórmula alternativa:\n');
fprintf('  x1 = %.15f (PRECISA!)\n', x1_alt);

% Verificação
verificacao = x1_alt^2 + 1000*x1_alt + 1;
fprintf('\nVerificação: x1^2 + 1000x1 + 1 = %.2e (deve ser ≈ 0)\n', verificacao);
```

Saída:

=== CANCELAMENTO CATASTRÓFICO ===

Problema: Calcular $(\sqrt{1+x} - 1) / x$ para x pequeno

Valor esperado: 0.5

x	Direto	Racionalizado	Diferença
1e-01	0.48809166415	0.48809166415	0.00e+00
1e-05	0.50000025000	0.50000025000	0.00e+00
1e-10	0.50000000000	0.50000000000	0.00e+00
1e-15	0.50000000000	0.50000000000	0.00e+00
1e-16	0.00000000000	0.50000000000	5.00e-01 ← ERRO!
1e-17	0.00000000000	0.50000000000	5.00e-01 ← ERRO!

Raízes de $ax^2 + bx + c = 0$

Equação: $x^2 + 1000x + 1 = 0$

Fórmula tradicional:

$x_1 = -0.0010000000000000$ (CANCELAMENTO!)

$x_2 = -999.9990000000000000$

Fórmula alternativa:

$x_1 = -0.001000001000001$ (PRECISA!)

Verificação: $x_1^2 + 1000x_1 + 1 = -8.88e-16$ (deve ser ≈ 0)

3.10. Exercícios Propostos

- Defina erro absoluto e erro relativo com suas próprias palavras.
- Qual é a diferença entre acurácia e precisão? Dê exemplos.
- Explique o que é erro de truncamento. Dê um exemplo com série de Taylor.
- Calcule o erro absoluto e relativo: valor exato = 10, valor aproximado = 9.9
- A que se deve o erro de arredondamento em computadores?
- Represente 0.1 em binário (pelo menos 8 bits). Por que não é exato?
- Explique por que a subtração de números muito próximos causa problemas numéricos.
- Demonstre que para uma série alternada, o erro de truncamento é menor que o primeiro termo ignorado.
- Calcule a propagação de erros para: $z = x/y$ onde $x = 10 \pm 0.1$ e $y = 5 \pm 0.05$

- j. Um método numérico converge para 2.71828, mas o algoritmo foi feito para calcular π . Qual conceito falha: acurácia ou precisão?
- k. Compare a série de Maclaurin para $\cos(x)$ usando 5, 10 e 15 termos. Mostre como o erro diminui.